

Module 4

Configure code scanning on GitHub



Agenda



GitHub Foundations Part 1

- 1 Introduction to Git ✓
- 2 Introduction to GitHub ✓
- 3 Introduction to GitHub's products ✓
- 4 Configure code scanning on GitHub
- 5 Introduction to GitHub Copilot ✓
- 6 Code with GitHub Codespaces ✓
- 7 Manage your work with GitHub Projects
- 8 Communicate effectively on GitHub using Markdown

*git init
git commit -m
git push*

GitHub Foundations Part 2

- 9 Contribute to an open-source project on GitHub
- 10 Manage an InnerSource program by using GitHub
- 11 Maintain a secure repository by using GitHub best practices
- 12 Introduction to GitHub administration
- 13 Authenticate and authorize user identities on GitHub
- 14 Manage repository changes by using pull requests on GitHub
- 15 Search and organize repository history by using GitHub
- 16 Using GitHub Copilot with Python

Introduction

This module introduces you to code scanning and its features. You'll learn how to implement code scanning using CodeQL, third-party tools, and GitHub Actions.

You'll also learn about the different ways you can configure code scanning to optimize your experience.

In this module, you learn how to:

- Describe code scanning.
- List the steps for enabling code scanning in a repository.
- Implement CodeQL analysis in a GitHub Actions workflow and (CI) tools.
- Explain how to configure code scanning on a repository using triggering events.
- Contrast the frequency of code scanning workflows (scheduled vs triggered by events).

What is code scanning?



Code Scanning with CodeQL

- **Purpose:** Analyzes code to find security vulnerabilities and coding errors, displaying alerts in the repository's Security tab.
- Default setup for quick configuration, advanced setup for customizable workflows, and running CodeQL CLI.
- Includes C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, and Swift.

FORTRAN?

LISP



Enabling CodeQL

- **Setup:** Navigate to the repository's Security tab, select Set up code scanning, and choose Default to enable CodeQL.
- **Triggers:** Code scanning runs automatically on push to protected branches or pull requests against the default branch.
- **Advanced Setup:** Generates a workflow file for more advanced configuration and additional steps.



Billing for Actions

- **Usage:** Code scanning uses GitHub Actions, consuming minutes for each run.
- **Free Usage:** Free for public repositories and self-hosted runners; private repositories have limited free minutes and storage.
- **Spending Limits:** Monthly billed accounts have a default spending limit of 0 USD, while invoiced accounts have unlimited spending limits.

Enable code scanning with third party tools

Consteaux

SARIF File Uploads for Code Scanning

- **Purpose:** Allows uploading of SARIF files generated outside GitHub to display code scanning alerts in the repository.
- Upload results using the code-scanning API, CodeQL CLI, or GitHub Actions.

Code-Scanning API

- **Functionality:** Retrieve and update information on code scanning alerts, analyses, and configurations.
- Access via HTTPS, using custom media types like application/sarif+json for specific endpoints.

CodeQL CLI

- Purpose: Analyze code and generate SARIF files for upload to GitHub.
- Components: Includes the CodeQL CLI product, queries, libraries, and precompiled queries for compatibility and performance.

GitHub Actions for SARIF Uploads

- Workflow: Use the upload-sarif action in a GitHub Actions workflow to upload SARIF files.
- Configuration: Set input parameters like sarif-file and configure the action to run on push or scheduled events.

KQL ← Azure Copilot
Kusto Query Lang

Microsoft Graph API → EntraID  email  SP

Azure Resource Graph API 

Sentinel SIEM

Configure code scanning

Configuring Code Scanning

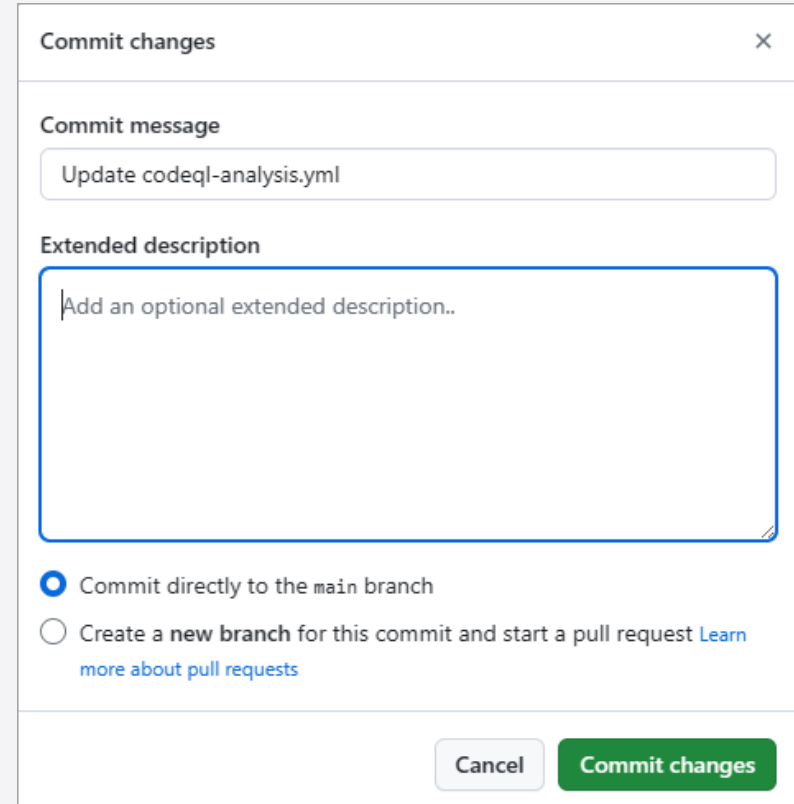
- Choose your own configuration to save time and set the best frequency for scans.

Editing Code-Scanning Workflow

- Workflow files are stored in the `.github/workflows` directory.
- Common Edits: Adjust scan frequency, specify languages or directories, and configure what CodeQL looks for in your code.

Scan Triggers

- Push Events: Default setup triggers scans on every push to the default or protected branches.
- Pull Requests: Scans are triggered on pull requests targeted against the default branch, with results appearing as alerts in pull-request checks.



The screenshot shows a 'Commit changes' dialog box with a close button (X) in the top right corner. It contains a 'Commit message' text field with the text 'Update codeql-analysis.yml'. Below this is an 'Extended description' text area with a placeholder text 'Add an optional extended description..'. At the bottom, there are two radio button options: 'Commit directly to the main branch' (which is selected) and 'Create a new branch for this commit and start a pull request' (with a link to 'Learn more about pull requests'). At the very bottom right are 'Cancel' and 'Commit changes' buttons.

Commit changes

Commit message

Update codeql-analysis.yml

Extended description

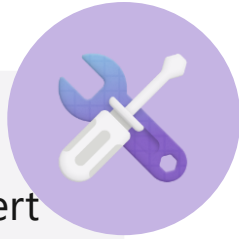
Add an optional extended description..

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Commit changes

Configure code scanning (cont.)



- **Pull Request Check Failures:** Customize alert severity levels that trigger pull request check failures in the repository settings under *Code security and analysis*.
- **Avoiding Unnecessary Scans:** Use paths-ignore in the code-scanning workflow to prevent scans on specific file changes, and adjust the cron value to modify the scanning schedule.

Code scanning

Automatically detect common vulnerabilities and coding errors.

Tools

CodeQL analysis
Identify vulnerabilities and errors with [CodeQL](#) for [eligible](#) repositories.

...

Other tools
Add any third-party code scanning tool.

[Explore workflows](#)

Protection rules

Check runs failure threshold
Select the alert severity level for code scanning check runs to fail. [Create a branch ruleset](#) to prevent a branch from merging when these checks fail.

Security: **High or higher** ▾

Other: **Only errors** ▾

Se

Re

Git

None

Only critical

✓ High or higher

Medium or higher

Any

or other tokens.

ts in public repositories. [Learn more about partner patterns.](#)

Disable

Disable

Exercise - Configure code scanning



This exercise checks your knowledge on configuring code scanning for your repository.

This exercise walks you through the following steps:

- How to enable code scanning and identify SQL injection vulnerabilities with CodeQL.
- How to build a secure software development pipeline that allows you to identify and prevent new security vulnerabilities from being introduced into your production code.

 [Start course](#)

Summary

This module introduced you to code scanning and its features. You learned how to implement code scanning using CodeQL, third-party tools, and GitHub Actions.

In this module, you learned how to:

- Describe code scanning.
- List the steps for enabling code scanning in a repository.
- Implement CodeQL analysis in a GitHub Actions workflow and (CI) tools.
- Explain how to configure code scanning on a repository using triggering events.
- Contrast the frequency of code scanning workflows (scheduled vs triggered by events).

End of presentation



